Problem & Data

The AI Backbone

From Vague Goal to Trained Model in 10 Steps

The 10-Step Roadmap

Each step builds on the previous one. You cannot skip steps — but you WILL loop back as you learn more about your data.

Step 1: Understand the Problem

Don't start with 'which model?' — Start with 'what would a HUMAN do?'

- Someone gives you a vague goal ("Build AI for hospital")
- You ask: "What SPECIFICALLY do you want to know?"
- You watch what a HUMAN expert actually does in that role
- You list each specific task the human performs
- Each task = one potential ML problem to solve
- Some tasks need ML, others are simple rules — identify which is which

Vague Goal	Specific Question	Tasks Found
Build AI for hospital	Predict which patients will need ICU?	Vital sign monitoring, Risk scoring, Triage
Make exam fair	Catch cheaters automatically?	Phone detection, Gaze tracking, Identity
Help cricket team	Predict match outcome?	Player stats, Weather, Pitch analysis

KEY INSIGHT: The #1 mistake beginners make is jumping straight to models. Spend 80% of your thinking time understanding the problem BEFORE touching code.

ExamGuard — Watching the Invigilator

What Invigilator Does	Data Used	How They Think	ML Type
Spots phone on desk	Images (camera)	"I recognize that shape" = trained on examples	YOLO (Supervised)
Notices student looking sideways repeatedly	Video (face direction)	"Head turned toward neighbor" = recognizing behavior	CNN + Pose Estimation
Senses something "off" about a student	Video (body language)	"Can't explain but not normal" = gut feeling	Autoencoder (Anomaly Detection)
Decides to confront or wait	Judgment (context)	"Is it worth disrupting? Balancing risk vs. harm"	Reinforcement Learning
Counts absent students	Counting	"30 desks, 28 present" = simple arithmetic	NOT ML — Simple Rule!

KEY INSIGHT: 1 project = 5 ML models + 1 simple rule. Not everything needs ML! A real AI system is a COLLECTION of models, each handling one specific task.

Step 2: The 3 Questions That Decide Your ML Type

How to use the 3 Questions:

1. Start with your specific task (e.g., "detect phones")

2. Q1 — What data will you feed the model?
   Images from CCTV? Text from reviews?
   Spreadsheet numbers? Audio clips?

3. Q2 — How does a human THINK about this?
   "I RECOGNIZE that" → Classification
   "That seems ABNORMAL" → Anomaly Detection
   "These GROUP together" → Clustering
   "I PREDICT this will happen" → Regression
   "I'm BALANCING tradeoffs" → RL

4. Q3 — Do you have labeled examples?
   Yes (labeled data) → Supervised Learning
   No (just raw data) → Unsupervised Learning
   Rewards/penalties → Reinforcement Learning

Q1 + Q2 + Q3 → Your ML Type

Use this table as a quick-reference to map ANY problem to the right approach.

Data (Q1)	Thinking (Q2)	Labels? (Q3)	ML Type	Model Examples
Images	Recognizing	Yes	Supervised	CNN / YOLO
Images	Something off	No	Anomaly Detection	Autoencoder
Text	Categorizing	Yes	Supervised	Naive Bayes / BERT
Numbers	Predicting category	Yes	Classification	LogReg / DT / RF
Numbers	Predicting amount	Yes	Regression	Linear / Poly / RF
Numbers	Something off	No	Anomaly Detection	Isolation Forest
Numbers	Grouping	No	Clustering	K-Means / DBSCAN
Any	Balancing tradeoffs	Rewards	Reinforcement Learning	Q-Learning / PPO

TIP: Most real projects need MULTIPLE rows from this table. ExamGuard uses rows 1, 2, and 8!

Practice: Apply 3 Questions to Real Problems

Restaurant Manager — "Reduce food waste"

Task	Data (Q1)	Thinking (Q2)	ML Type
Predict daily demand	Sales numbers	Predicting amount	Regression
Spot spoiled ingredients	Images	Something off	Anomaly
Group similar dishes	Menu + sales	Grouping	Clustering
Categorize complaints	Text reviews	Categorizing	Classification
Optimize pricing	Sales + prices	Balancing tradeoffs	RL

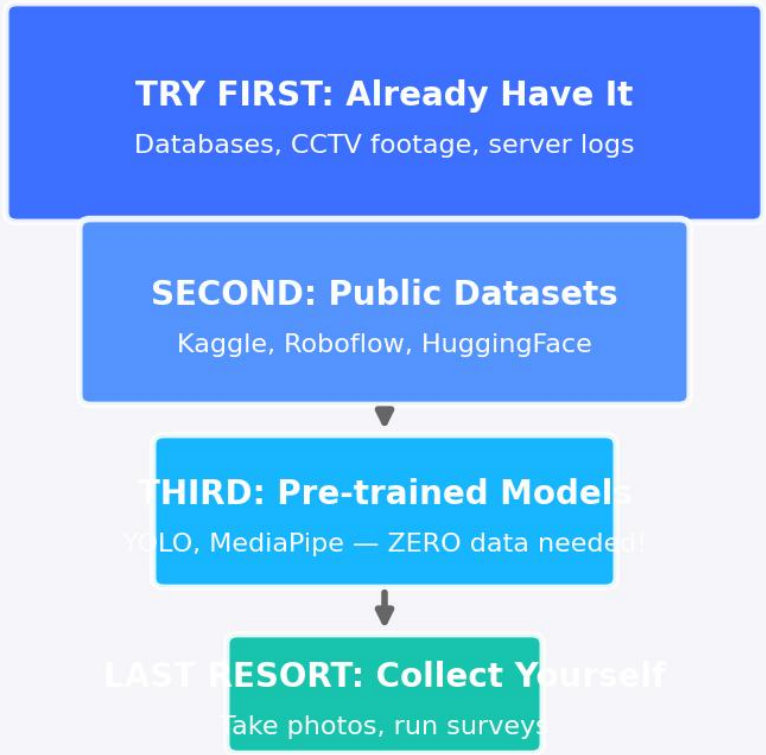
Clothing Store — "Reduce returns"

Task	Data (Q1)	Thinking (Q2)	ML Type
Predict if item returned	Purchase data	Predicting category	Classification
Recommend right size	Body + item data	Predicting amount	Regression
Detect listing errors	Product images	Something off	Anomaly
Group return reasons	Text feedback	Grouping	Clustering
Optimize return policy	Cost + satisfaction	Balancing tradeoffs	RL

NOTICE THE PATTERN:

Every real-world project decomposes into 4-6 sub-tasks. Each sub-task maps to a different ML type. This is why Step 1 (decomposition) is so critical — without it, you'd try to build ONE model to do everything, and it would fail.

Step 3: Where to Get Data



Data Source Priority (try top-to-bottom):

1. YOUR OWN DATA is always best because it matches your exact situation. CCTV from YOUR exam hall, patient records from YOUR hospital.

2. PUBLIC DATASETS are great for starting out. Kaggle.com has 200K+ datasets. Roboflow has labeled image datasets. HuggingFace for text.

3. PRE-TRAINED MODELS already know how to do common tasks. YOLO detects 80 objects out of the box. MediaPipe tracks faces and poses. You may need ZERO data!

4. COLLECTING YOUR OWN is expensive and slow. Only do this when nothing else works. But the result is the most valuable data.

Pre-trained Models — Start with ZERO Data

Model	What It Does	Data Needed	Install Command
YOLOv8	Detect 80 objects (phone, person, bag...)	Zero!	pip install ultralytics
MediaPipe Face	Face landmarks + gaze direction	Zero!	pip install mediapipe
MediaPipe Pose	Body skeleton tracking (33 points)	Zero!	(same package)
BERT / DistilBERT	Text understanding, sentiment, classification	Zero for basic use	pip install transformers
Whisper	Speech-to-text (98 languages)	Zero!	pip install openai-whisper

THE 10-PHOTO TEST — Before committing to any model:

1. Take 10 photos/samples from YOUR actual setup (your classroom, your camera angle)
2. Run the pre-trained model on those 10 samples
3. Check accuracy: >85% = Use AS-IS | 60-85% = FINE-TUNE with your data | <60% = Wrong model, try another
4. This takes 30 minutes and saves you WEEKS of wrong-path work

Step 4: Validate Your Data — 5 Checks

1. SIZE — Count your rows/images

Less than 500 samples is risky and may not generalize. 500-5,000 is workable for most tasks. 10,000+ is great. For images, even 200 can work with augmentation + transfer learning.

2. LOOK — Open 30 random samples

Are images clear enough? Are labels actually correct? Do the numbers make sense? You'd be surprised how many datasets have wrong labels or corrupted files.

3. RELEVANCE — Does it match YOUR situation?

A dataset of phones on streets is NOT the same as phones on exam desks. Different angles, lighting, and context. Your model will fail if training data doesn't match deployment.

4. BALANCE — Check class distribution

500 phone images + 500 no-phone images = GOOD. 4,500 no-phone + 50 phone = BAD. The model will just say 'no phone' every time and be 98% accurate but useless.

5. TRUST — Who made this dataset?

Check: download count, community ratings, recent updates, clear documentation. A dataset with 50K downloads and 4.8 stars is safer than one uploaded yesterday with no description.

RED FLAGS: No documentation | Last updated 5+ years ago | Less than 100 downloads | Mismatch with your use case | Only 1 class

Step 5: Clean Your Data — 80% of ML Work

Problem	Example	Fix
Missing values	50% of rows have blank fields	Fill with average/median, or delete row if too many missing
Duplicates	Same row appears 100 times	Find duplicates and remove (keep one copy)
Wrong labels	"Phone" label on a no-phone image	Manually check 50 random samples, fix incorrect ones
Outliers	Age = 500, Salary = -10	Cap at reasonable max/min, or remove if clearly error
Wrong format	"Rs 50,000" instead of 50000	Convert strings to numbers, standardize currency
Inconsistent	"Male", "M", "m", "male"	Map all variants to one standard format
Blurry images	Can't see what's in the photo	Remove or recapture. Blurry = noise for the model.

GARBAGE IN = GARBAGE OUT

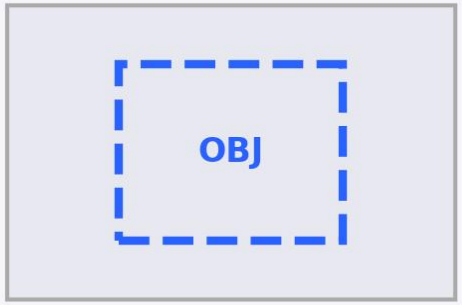
The single most important rule in ML. A simple model on clean data will ALWAYS beat a fancy model on dirty data. Data scientists spend 60-80% of their time on cleaning alone.

Step 6: Label Your Data

Classification

"phone"

Bounding Box



x, y, w, h

Segmentation

pixel mask

Text Label

"positive"

Best Practices for Labeling:

- Be CONSISTENT — same rules for every image. Write a labeling guide.
- Label TIGHTLY — bounding box should hug the object, not include extra space.
- When in doubt, SKIP the image rather than guess wrong.
- Have 2 people label independently, then compare. Agreement < 90%? Rules are unclear.

Tools: Roboflow (web, free tier) | LabelImg (desktop, free) | CVAT (open source) | Label Studio

Step 7: Data Augmentation — Make Small Data Bigger

1 image × 6 techniques = 6 images. 200 originals → 1,200 augmented!

When TO augment:

- You have fewer than 1,000 images per class
- Your model is overfitting (training accuracy high, test accuracy low)
- You want your model to handle different lighting/angles

When NOT to augment:

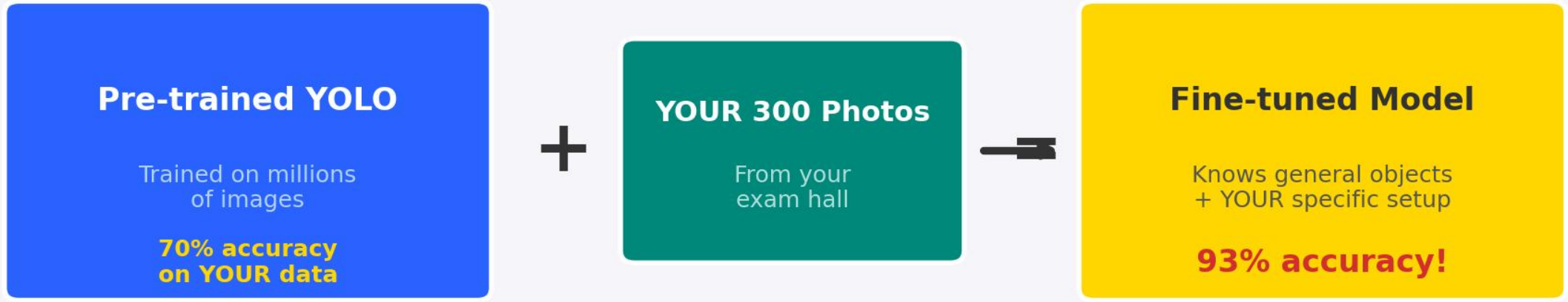
- Text data (flipping text makes no sense)
- Numbers/tabular data (use SMOTE instead)
- If augmented images become unrealistic

Tools for Augmentation:

- Roboflow — web-based, automatic augmentation pipeline
- Albumentations — Python library, 60+ transforms
- torchvision.transforms — built into PyTorch
- Keras ImageDataGenerator — built into TensorFlow

Common transforms: flip, rotate, brightness, contrast, crop, blur, noise, scale, shear, cutout

Step 8: Transfer Learning — Build on What Exists



Extending Your Model in Phases:

Phase 1: Phones only → Train on 300 phone photos → Deploy & test

Phase 2: + Cheat sheets → Add 200 chit photos, retrain → Now detects 2 things

Phase 3: + Earpieces → Add 150 earpiece photos → Now detects 3 things

Phase 4: + Smartwatches → Add 100 watch photos → Full detection system

KEY RULE: You NEVER train from scratch. Always build on what exists.

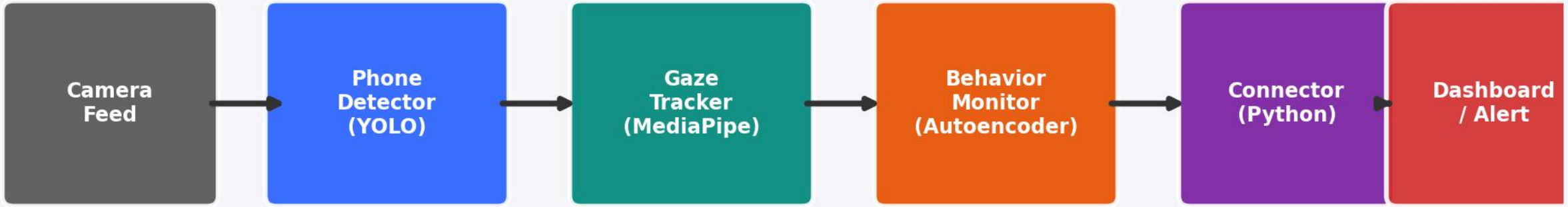
Transfer learning = standing on the shoulders of giants.

Step 9: The Complete Pipeline

Where Your Time Actually Goes

Notice: 60% of the work is data preparation (Steps 3-8). Only 20% is actual model training. This is why 'data engineering' is a separate career. Most beginners spend 90% on models and 10% on data — and then wonder why their models don't work.

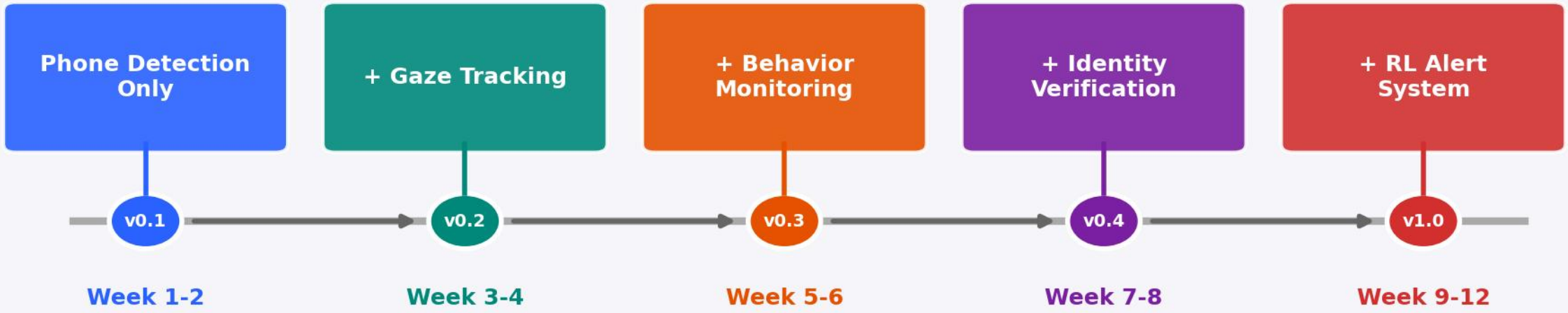
Step 10: How Real AI Systems Work



Key Rules for Multi-Model Systems:

- Each model runs INDEPENDENTLY — if one fails, others keep working
- The Connector is SIMPLE Python code (if/else logic), not another ML model
- Models communicate through standard formats: JSON, CSV, or simple variables
- Build and test EACH model separately before connecting them
- Use confidence scores: "YOLO says phone detected with 87% confidence"
- Set thresholds per model: only alert if confidence > 70% (configurable)
- Log everything — when models disagree, logs help you debug

Build ONE Module at a Time



Why Build Incrementally?

- Each version is DEPLOYABLE and TESTABLE on its own
- You get feedback from real users after Week 2, not Week 12
- If v0.1 doesn't work, you haven't wasted 12 weeks
- Each new module builds on proven working infrastructure
- Your stakeholders see progress every 2 weeks (keeps funding alive!)

Quick Reference — 10 Steps to AI

1. **UNDERSTAND** the problem — watch what a human does, list every specific task.
2. **ASK 3 QUESTIONS** — What data? How thinking? Past examples? → ML type.
3. **FIND DATA** — check what you have, then public datasets, then pre-trained models.
4. **VALIDATE DATA** — check size, look at samples, relevance, balance, trust.
5. **CLEAN DATA** — fix missing values, duplicates, wrong labels, outliers, formats.
6. **LABEL DATA** — classification, bounding box, or segmentation. Be consistent.
7. **AUGMENT DATA** — flip, rotate, brightness. 200 images → 1,200 images.
8. **TRANSFER LEARN** — start from pre-trained model, fine-tune with YOUR data.
9. **BUILD PIPELINE** — Problem → Data → Clean → Train → Test → Deploy. 60% = data work.
10. **CONNECT MODULES** — each model is independent, simple Python connector merges outputs.

Remember: 80% thinking, 20% coding. The best AI engineers are problem-solvers first, coders second.